

CubeSLAM: Monocular 3-D Object SLAM

Shichao Yang¹ and Sebastian Scherer²

Abstract—In this paper, we present a method for single image three-dimensional (3-D) cuboid object detection and multiview object simultaneous localization and mapping in both static and dynamic environments, and demonstrate that the two parts can improve each other. First, for single image object detection, we generate high-quality cuboid proposals from two-dimensional (2-D) bounding boxes and vanishing points sampling. The proposals are further scored and selected based on the alignment with image edges. Second, multiview bundle adjustment with new object measurements is proposed to jointly optimize poses of cameras, objects, and points. Objects can provide long-range geometric and scale constraints to improve camera pose estimation and reduce monocular drift. Instead of treating dynamic regions as outliers, we utilize object representation and motion model constraints to improve the camera pose estimation. The 3-D detection experiments on SUN RGBD and KITTI show better accuracy and robustness over existing approaches. On the public TUM, KITTI odometry and our own collected datasets, our SLAM method achieves the state-of-the-art monocular camera pose estimation and at the same time, improves the 3-D object detection accuracy.

Index Terms—Dynamic SLAM, object detection, object simultaneous localization and mapping (SLAM), SLAM.

I. INTRODUCTION

OBJECT detection and simultaneous localization and mapping (SLAM) are two important tasks in computer vision and robotics. For example, in autonomous driving, vehicles need to be detected in three-dimensional (3-D) space in order to remain safe. In augmented reality, 3-D objects also need to be localized for more realistic physical interactions. Different sensors can be used for these tasks, such as laser-range finders, stereo, or RGB-D cameras, which can directly provide depth measurement. Alternatively, monocular cameras are attractive because of their low cost and small size. Most of the existing monocular approaches solve object detection and SLAM separately, while they also depend on prior object models which limit their application to general environments. Therefore, we focus on the general 3-D object mapping problem by solving object

detection and multiview object SLAM jointly in both static and dynamic environments.

For object detection, many algorithms are able to detect different 2-D objects with various size and viewpoints in large datasets using convolutional neural networks (CNNs) [1]. 3-D object objection is more challenging and has also attracted attention recently, such as for vehicle detection [2], [3]. For SLAM or structure from motion (SfM), the classic approach is to track visual geometric features, such as points [4], lines [5], planes [6] across frames then minimize the reprojection or photometric error through bundle adjustment (BA). However, apart from these low-level features in the environments, objects are also important components which have not been well-explored in SLAM. Detecting and mapping 3-D objects can greatly improve robot intelligence for environment understanding and human-robot interaction. In addition, objects used as SLAM landmarks can also provide additional semantic and geometric constraints to improve camera pose estimation.

Most existing SLAM approaches assume the environment to be static or mostly static. Features from dynamic regions are often treated as outliers and not used for camera pose estimation [4], however this assumption may not hold in many practical environments. For example, there are many moving vehicles and pedestrians on the road. It is also an important task to detect and predict the trajectory of moving objects in many applications.

In this work, we propose a system to combine 2-D and 3-D object detection with SLAM pose estimation, together for both static and dynamic environments shown in Fig. 1. Given the detected 2-D object, many 3-D cuboid proposals are efficiently generated through vanishing point (VP) sampling, under the assumptions that the cuboid will fit 2-D bounding box tightly after projection. Then the selected cuboid proposals are further optimized with points and cameras through multiview BA. Objects are utilized in two ways: to provide geometry and scale constraints in BA, and to provide depth initialization for points difficult to triangulate. The estimated camera poses from SLAM are also used for single-view object detection. Finally, in the dynamic case, instead of treating moving objects as outliers, we jointly optimize the trajectories of camera and objects based on dynamic point observation and motion model constraints. In summary, our contributions are as follows.

- An efficient, accurate, and robust single image 3-D cuboid detection approach without requiring prior object models.
- An object SLAM method with novel measurements between cameras, objects, and points, that achieves better pose estimation on many datasets, including KITTI benchmark.

Manuscript received January 13, 2019; accepted March 21, 2019. Date of publication May 7, 2019; date of current version August 1, 2019. This work was supported by the Amazon Research Award #2D-01038138. This paper was recommended for publication by Associate Editor J. M. M. Montiel and Editor F. Chaumette upon evaluation of the reviewers' comments. (Corresponding author: Shichao Yang.)

The authors are with the Robotics Institute, Carnegie Mellon University, Pittsburgh, PA 15213-3890 USA (e-mail: shichaoy@andrew.cmu.edu; 2013ysc@gmail.com; basti@andrew.cmu.edu).

This paper has supplementary downloadable multimedia material available at <http://ieeexplore.ieee.org> provided by the authors. This includes a video file in standard MPEG-4 video format, which shows the experimental results of "CubeSLAM: Monocular 3D Object SLAM"

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TRO.2019.2909168

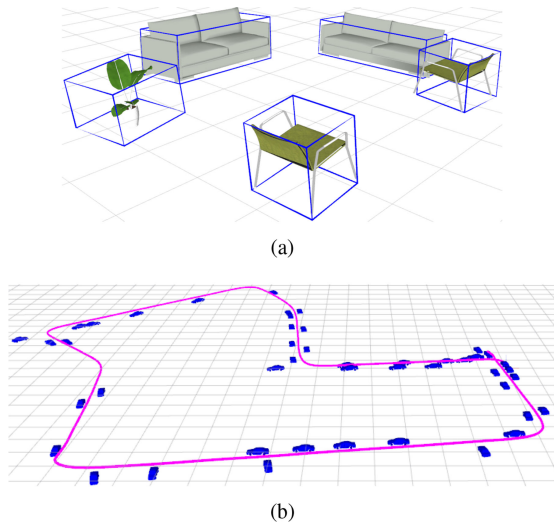


Fig. 1. Monocular 3-D object detection and mapping without requiring prior object models. Mesh model is just for visualization and not used for detection. (a) ICL NUIM data with various objects, whose position, orientation, and dimension are optimized by SLAM. (b) KITTI 07. With object constraints, our monocular SLAM can build a consistent map and reduce scale drift, without loop closure and constant camera height assumption.

- Results demonstrating that object detection and SLAM benefit each other.
- A method to utilize moving objects to improve pose estimation in dynamic scenarios.

This paper is organized as follows. We first introduce the single image detection in Section III, then explain the object SLAM in static (Section IV) and dynamic environments (Section V), followed by implementations in Section VI and experiments in Section VII and Section VIII. Part of the code is publicly available.¹ Section IX concludes this paper.

II. RELATED WORK

A. Single Image 3-D Object Detection

3-D object detection from a single image is much more challenging compared to 2-D because more object pose variables and the camera projective geometry need to be considered. Existing 3-D detection approaches can be divided into two categories: With or without shape priors, such as CAD models. With prior models, the best object pose to align with RGB images can be found through keypoint perspective-n-point (PnP) matching [7], hand-crafted texture features [8], or more recent deep networks [3], [9].

Without prior models, objects are usually represented by cuboids. The typical approach is to combine geometry modeling with learning. For example, objects can be generated by a combination of Manhattan edges or rays through VPs [10], [11]. Chen *et al.* proposed to exhaustively sample many 3-D boxes on the ground, and then select based on various context features [12]. Two works similar to ours are [13] and [14], which used projective geometry to find cuboids to fit the 2-D bounding box tightly. We extend it to work without prediction of object size and orientation.

¹[Online]. Available: https://github.com/shichaoy/cube_slam

B. Multiview Object SLAM

There are many point-based visual SLAM algorithms, such as ORB SLAM [4] and DSO [15], which can achieve impressive results in general environments. Object-augmented mapping is also explored in recent years. There are typically two categories of them, either decoupled or coupled. The decoupled approaches first build a SLAM point cloud map and then further detect and optimize 3-D object poses based on point cloud clustering and semantic information [16]–[18]. It showed improved results compared to 2-D object detections, however it did not change the SLAM part; thus the decoupled approach may fail if SLAM cannot build a high quality map.

The coupled approach is usually called object-level SLAM. Bao *et al.* proposed the first Semantic SfM to jointly optimize camera poses, objects, points, and planes [19]. Salas *et al.* [20] proposed a practical SLAM system called SLAM++ using RGB-D cameras and prior object models. Frost *et al.* represented objects as spheres to correct the scale drift of monocular SLAM [21], similarly in [22]. Recently, a real-time monocular object SLAM using the prior object models was proposed in [23]. Rubino *et al.* [24] solved multiview 3-D ellipsoid object localization analytically and QuadriSLAM [25] extended it to an online SLAM without prior models. Uncertain data association of object SLAM is addressed in [26]. Yang *et al.* [27] proposed a similar idea to combine scene understanding with SLAM, but only applied to planes.

Recently, there is also some end-to-end deep learning based SLAM without object representations, such as DVSO [28] and DeepVO [29]. They have achieved great performance on KITTI datasets, however, it is still unclear if they would generalize to novel environments.

C. Dynamic Environment SLAM

SLAM in dynamic environments has been a challenging problem. Most existing approaches treat dynamic region features as outliers and only utilize static background for pose estimation [4], [30], [31]. After the static SLAM problem is solved, some other works additionally detect, track, and optimize the trajectory of dynamic objects in order to build a complete 3-D map [14], [32], [33]. The optimization is based on the object's re-projection error, object motion model, and also the point feature observations on the object. However, these approaches are likely to fail in highly dynamic environments due to the lack of reliable static background features.

There is a recent work utilizing dynamic point BA to improve camera pose estimation, based on the rigid shape and constant motion assumption [34]; however, the paper showed limited real dataset results and did not explicitly represent objects in the map.

III. SINGLE IMAGE 3-D OBJECT DETECTION

A. 3-D Box Proposal Generation

1) *Principles:* Instead of randomly sampling object proposals in 3-D space, we utilize the 2-D bounding box to efficiently generate 3-D cuboid proposals. A general 3-D cuboid can be represented by 9 DoF parameters: 3 DoF position $\mathbf{t} = [t_x, t_y, t_z]$, 3 DoF rotation R , and 3 DoF dimension $\mathbf{d} = [d_x, d_y, d_z]$. The

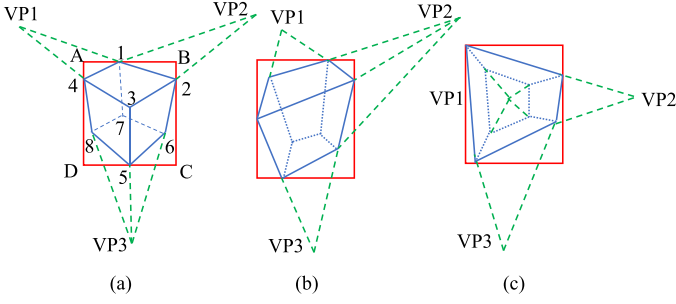


Fig. 2. Cuboid proposals generation from 2-D object box. If VP and one corner are estimated, the other seven corners can also be computed analytically. For example in (a), given corner 1, corners 2 and 4 can be determined through line intersection, and same as other corners.

cuboid coordinate frame is built at the cuboid center, aligned with the main axes. The camera intrinsic calibration matrix K is also known. Based on the assumptions that the cuboid's projected corners should fit the 2-D bounding box tightly, there are only four constraints corresponding to four sides of the 2-D box; therefore, it is not possible to fully constrain all nine parameters. Other information is needed, for example, the provided or predicted object dimensions and orientations used in many vehicle detection algorithms [12]–[14]. Rather than relying on the predicted dimensions, we utilize the VP to change and reduce the regression parameters in order to work for general objects.

The VP is the parallel lines' intersection after projection onto perspective images [35]. A 3-D cuboid has three orthogonal axes and can form three VPs after projections depending on object rotation R wrt camera frame and calibration matrix K

$$VP_i = K R_{col(i)}, \quad i \in \{1, 2, 3\} \quad (1)$$

where $R_{col(i)}$ is the i th column of R .

2) *Get 2-D Corners From the VP*: We first show how to get eight 2-D cuboid corners based on the VP. Since at most three cuboid faces can be observed simultaneously, we can divide the cuboid configurations into three common categories based on the number of observable faces shown in Fig. 2. Each configuration can be left-right symmetric. Here we explain Fig. 2(a) in more detail. Suppose three VPs and top corner p_1 are known or estimated, and $\times$ represents the intersection of two lines, then $p_2 = (\overline{VP_1, p_1}) \times (\overline{B, C})$, similarly for p_4 . $p_3 = (\overline{VP_1, p_4}) \times (\overline{VP_2, p_2})$, $p_5 = (\overline{VP_3, p_3}) \times (\overline{C, D})$, similarly for the remaining corners.

3) *Get 3-D Box Pose From 2-D Corners*: After we get the cuboid corners in 2-D image space, we need to estimate the cuboid's 3-D corners and pose. We divide the objects into two scenarios.

Arbitrary pose objects: we use PnP solver to solve the general cuboid's 3-D position and dimensions up to a scale factor due to the monocular scale ambiguity. Mathematically, the cuboid's eight 3-D corners in the object frame are $[\pm d_x, \pm d_y, \pm d_z] / 2$ and in the camera frame are $R[\pm d_x, \pm d_y, \pm d_z] / 2 + \mathbf{t}$. As shown in Fig. 2(a), we can select four adjacent corners, such as 1, 2, 4, 7, which can be projected from the abovementioned 3-D corners; for example; corner 1

$$p_1 = \pi(R[d_x, d_y, d_z] / 2 + \mathbf{t}). \quad (2)$$

π is the camera projective function and p_i ($i = 1, 2, \dots, 8$) is one of the eight 2-D object corners. Each corner provides two constraints, thus four corners can fully constrain the object pose (9 Dof) except the scale. Iterative or noniterative PnP solvers can be found in [35].

Ground objects: for ground objects lying on the ground plane, we can further simplify the abovementioned process and get the scale factor more easily. We build the world frame on the ground plane then object's roll/pitch angle is zero. Similar to the previous section, we can get eight 2-D corners from VP. Then instead of using the complicated PnP solver in (2), we can directly back project ground corner pixels to the 3-D ground plane and subsequently compute other vertical corners to form a 3-D cuboid. This approach is very efficient and has analytical expressions. For example for corner 5 on the 3-D ground plane expressed by $[\mathbf{n}, m]$ (normal and distance in camera frame), the corresponding 3-D corner $\mathbf{P}_5$ is the intersection of backprojected ray $K^{-1}p_5$ with the ground plane

$$\mathbf{P}_5 = \frac{-m}{\mathbf{n}^\top (K^{-1}p_5)} K^{-1}p_5. \quad (3)$$

Similarly, a more detailed projection process is explained in [27]. The scale is determined by the camera height in the projection process.

4) *Sample VP and Summary*: From the analysis in the previous two sections, the box estimation problem changes to how to get three VPs and one top 2-D corner, because after we get the VP, we can use Section III-A2 to compute 2-D corners, then use Section III-A3 to compute 3-D box. From (1), VP is determined by object rotation matrix R . Though deep networks can be used to directly predict them with large amounts of data training, we choose to sample them manually, and then score (rank) them for the purpose of generalizability.

For general objects, we need to sample the full rotation matrix R , however, for ground objects, camera's roll/pitch and object's yaw are used and sampled to compute R . More importantly, in datasets, such as SUN RGBD or KITTI, camera roll/pitch are already provided. For multiview video data, we use SLAM to estimate camera poses. Therefore the sampling space is greatly reduced and also becomes more accurate. In this paper's experiments, we only consider ground objects.

B. Proposal Scoring

After sampling many cuboid proposals, we define cost functions to score them as shown in Fig. 3. Different functions have been proposed, such as semantic segmentation [12], edge distance [8], HOG features [10]. We propose some fast and effective cost functions to align the cuboid with image edge features. This approach works best for “boxy with clear edge” objects, but also works decently well for bicycles and toilets, etc., as shown in later experiments due to constraints from VP and robust edge filtering. We first denote the image as I and cuboid proposal as $O = \{R, \mathbf{t}, \mathbf{d}\}$ defined in Section III-A1, then the cost function is defined as

$$E(O|I) = \phi_{\text{dist}}(O, I) + w_1 \phi_{\text{angle}}(O, I) + w_2 \phi_{\text{shape}}(O) \quad (4)$$

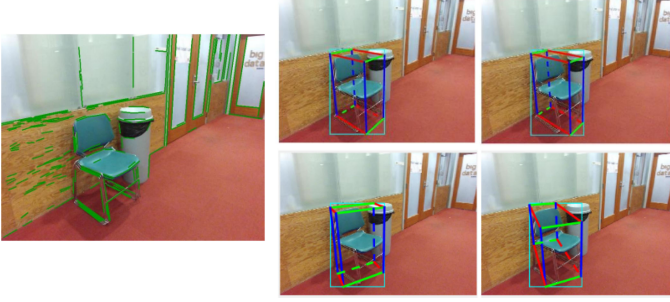


Fig. 3. Cuboid proposal scoring. (Left) Edges used to score the proposals. (Right) Cuboid proposals generated from the same 2-D cyan bounding box. The top left is the best and bottom right is the worst after scoring.

where ϕ_{dist} , ϕ_{angle} , ϕ_{shape} are three of costs which will be explained as follows. w_1 and w_2 are weight parameters between different costs. We set $w_1 = 0.8$, $w_2 = 1.5$ after manual search on small sample datasets.

1) *Distance Error* $\phi_{\text{dist}}(O, I)$: The 2-D cuboid edges should match with the actual image edges. We first detect Canny edges and build a distance transform map based on them. Then for each visible cuboid edge [solid blue line in Fig. 2(a)], we evenly sample ten points on it and summarize all the distance map value divided by the 2-D box's diagonal. This is similar to the Chamfer distance [10].

2) *Angle Alignment Error* $\phi_{\text{angle}}(O, I)$: The distance error is sensitive to the noisy false positive edges, such as object surface textures. Therefore, we also detect long line segments [36] (shown as the green lines in Fig. 3) and measure whether their angles align with VP. These lines are first associated with one of three VPs based on the point-line support relationship [11]. Then for each VP_i , we can find two outmost line segments with the smallest and largest slope, denoted as $\langle l_{i.ms}, l_{i.mt} \rangle$ and $\langle l_{i.ns}, l_{i.nt} \rangle$, respectively. $\langle a, b \rangle$ represents the slope angle of a line with two endpoints a, b . Finally the angle alignment error is

$$\phi_{\text{angle}}(O, I) = \sum_{i=1:3} \|\langle l_{i.ms}, l_{i.mt} \rangle - \langle \text{VP}_i, l_{i.mt} \rangle\| + \|\langle l_{i.ns}, l_{i.nt} \rangle - \langle \text{VP}_i, l_{i.nt} \rangle\|. \quad (5)$$

3) *Shape Error* $\phi_{\text{shape}}(O)$: The previous two costs can be evaluated efficiently just in 2-D image space. However, similar 2-D cuboid corners might generate quite different 3-D cuboids. We add a cost to penalize the cuboids with a large skew (length/width) ratio. More strict priors could be applied, for example, the estimated or fixed dimensions of specific types of objects.

$$\phi_{\text{shape}}(O) = \max(s - \sigma, 0) \quad (6)$$

where $s = \max(d_x/d_y, d_y/d_x)$ is the object skew ratio and σ is a threshold set be 1 in our experiments. If $s < \sigma$, no penalty is applied.

IV. OBJECT SLAM

We extend the single image 3-D object detection to multiview object SLAM to jointly optimize object pose and camera pose.

The system is built on feature point based ORB SLAM2 [4], which includes the front end of camera tracking and back end of BA. Our main change is the modified BA to include objects, points, and camera poses together, which will be explained in detail in this section. Other SLAM implementation details are given in Section VI-B. Static objects are used in this section and dynamic objects are addressed in the next section.

A. BA Formulation

BA is the process to jointly optimize different map components, such as camera poses and points [4] [15]. Points are also used in most of our experiments because objects alone usually cannot fully constrain camera poses. If we denote the set of camera poses, and 3-D cuboids and points as $C = \{C_i\}$, $O = \{O_j\}$, $P = \{P_k\}$, respectively, BA can be formulated as a nonlinear least squares optimization problem

$$C^*, O^*, P^* = \arg \min_{\{C, O, P\}} \sum_{C_i, O_j, P_k} \|e(c_i, o_j)\|_{\Sigma_{ij}}^2 + \|e(c_i, p_k)\|_{\Sigma_{ik}}^2 + \|e(o_j, p_k)\|_{\Sigma_{jk}}^2 \quad (7)$$

where $e(c, o)$, $e(c, p)$, and $e(o, p)$ represent the measurement error of camera object, camera point, object point, respectively. Σ is covariance matrix of different error measurements. Definitions of variables and errors are in the following. Then the optimization problem can be solved by Gauss–Newton or Levenberg–Marquardt algorithm available in many libraries, such as g2o [37] and iSAM [38].

Notations: Camera poses are represented by $T_c \in SE(3)$ and points are represented by $P \in \mathbb{R}^3$. As explained in Section III-A1, cuboid objects are modeled as 9 DoF parameters: $O = \{T_o, \mathbf{d}\}$ where $T_o = [R \ \mathbf{t}] \in SE(3)$ is 6 DoF pose, and $\mathbf{d} \in \mathbb{R}^3$ is the cuboid dimension. In some environments, such as KITTI, we can also use the provided object dimension, then $\mathbf{d}$ is not needed to optimize. Subscript m indicates the measurement. The coordinate system is shown in Fig. 3(b).

B. Measurement Errors

1) *Camera-Object Measurement*: We propose two kinds of measurement errors between objects and cameras.

a) *3-D Measurements*: The first is 3-D measurement utilized when the 3-D object detection is accurate, for example, if an RGBD sensor is used. The detected object pose $O_m = (T_{om}, \mathbf{d}_m)$ from single image detection in Section III-A serves as the object measurement from the camera frame. To compute its measurement error, we transform the landmark object to the camera frame and then compare it with the measurement

$$e_{co,3D} = [\log((T_c^{-1}T_o)T_{om}^{-1})_{\mathfrak{se}_3} \quad \mathbf{d} - \mathbf{d}_m] \quad (8)$$

where $\log$ maps the $SE(3)$ error into 6 DoF tangent vector space, therefore $e_{co,3D} \in \mathbb{R}^9$. Huber robust cost function is applied to all measurement errors to improve the robustness [4].

We need to note that, without prior object models, our image-based cuboid detection cannot differentiate between the front or back of objects. For example, we can represent the same cuboid

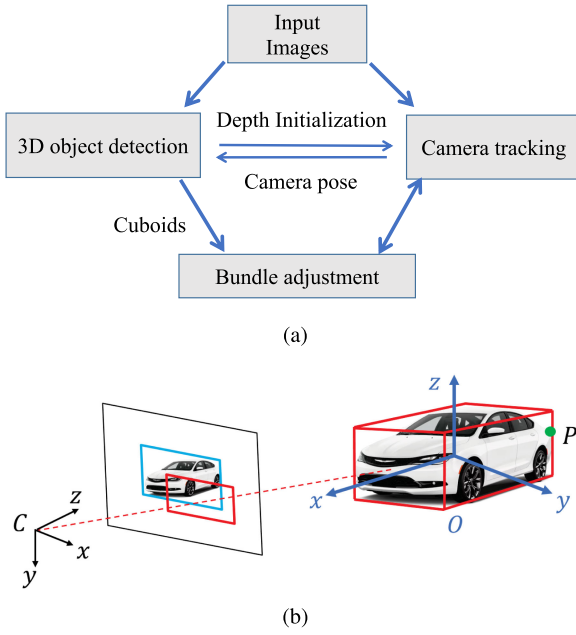


Fig. 4. (a) Our object SLAM pipeline. Single view object detection provides cuboid landmark and depth initialization for SLAM, while SLAM can estimate camera pose for more accurate object detection. (b) Showing the coordinate system and measurement errors between cameras, objects, and points during BA.

by rotating the object coordinate frame by 90° and swapping length with width value. Therefore, we need to rotate along the height direction for $0, \pm 90^\circ, 180^\circ$ to find the smallest error in (8).

b) 2-D Measurements: For the 2-D measurement, we project the cuboid landmark onto the image plane to get the 2-D bounding box shown as the red rectangle in Fig. 4(b) and then compare it with the blue detected 2-D bounding box. In more detail, we project the eight corners onto the image and find the min and max of the projected pixels' xy coordinates to form a rectangle

$$\begin{aligned} [u, v]_{\min} &= \min\{\pi(R[\pm d_x, \pm d_y, \pm d_z]/2 + \mathbf{t})\} \\ [u, v]_{\max} &= \max\{\pi(R[\pm d_x, \pm d_y, \pm d_z]/2 + \mathbf{t})\} \\ \mathbf{c} &= ([u, v]_{\min} + [u, v]_{\max})/2 \\ \mathbf{s} &= [u, v]_{\max} - [u, v]_{\min} \end{aligned} \quad (9)$$

where $[u, v]_{\min, \max}$ are the min/max xy coordinates of all eight projected corners, namely the top left and bottom right corners of projected rectangle. $\mathbf{c}$ and $\mathbf{s}$ are the center and size of the 2-D box, respectively. Both of them are 2-D vectors, therefore $[\mathbf{c}, \mathbf{s}] \in \mathbb{R}^4$. The 4-D rectangle error is then defined as

$$e_{co,2D} = [\mathbf{c}, \mathbf{s}] - [\mathbf{c}_m, \mathbf{s}_m]. \quad (10)$$

This measurement error has much less uncertainty compared to the 3-D error in (8) because 2-D object detection is usually more accurate compared to 3-D detection. This is similar to projecting map points onto images to formulate reprojection error. However it also loses information after projection because



Fig. 5. Object association in the dynamic and occluded scenarios of KITTI 07. Green points are the nonobject points, and points in other colors are associated with objects of the same color. The front cyan moving car is not added as a SLAM landmark as no feature point is associated with it. Points in object overlapping areas are not associated with any object due to ambiguity.

many different 3-D cuboids can project to the same 2-D rectangle, thus more observations are needed to fully constrain the camera poses and cuboids.

Modeling and estimating the error covariance Σ or hessian matrix W is not straightforward compared to points due to the complicated detection process. Therefore we simply give more weights to the semantic confident and geometric close objects. Suppose the cuboid-camera distance is d and the object's 2-D detection probability is p , then we can define $w = p \times \max(70 - d, 0)/50$ on KITTI data, where 70 m is truncation distance. Parameters may vary with different datasets.

2) Object-Point Measurement: Points and objects can provide constraints for each other. If point P belongs to an object shown in Fig. 4(b), it should lie inside the 3-D cuboid. So we first transform the point to the cuboid frame then compare with cuboid dimensions to get 3-D error

$$e_{op} = \max(|T_o^{-1}P| - \mathbf{d}_m, 0) \quad (11)$$

where $\max$ operator is used because we only encourage points to lie inside cuboid instead of exactly on surfaces.

3) Camera-Point Measurement: We use the standard 3-D point reprojection error in feature-based SLAM [4]

$$e_{cp} = \pi(T_c^{-1}P) - z_m \quad (12)$$

where z_m is the observed pixel coordinate of 3-D point P .

C. Data Association

Data association across frames is another important part of SLAM. Compared to point matching, object association seems to be easier as more texture is contained and many 2-D object tracking or template matching approaches can be used. Even 2-D box overlapping can work in some simple scenarios. However, these approaches are not robust if there is severe object occlusion with repeated objects as shown in Fig. 5. In addition, dynamic objects need to be detected and removed from current SLAM optimization, but standard object tracking approaches cannot classify whether it is static or not, unless specific motion segmentation is used.

We thus propose another method for object association based on feature point matching. For many point-based SLAM methods [4], feature points in different views can be effectively matched through descriptor matching and epipolar geometry

checking. Therefore we first associate feature points to their corresponding object if points are observed in the 2-D object bounding box for at least two frames and their 3-D distance to the cuboid center is less than 1 m. For example in Fig. 5, the feature points have the same color with their associated object. Note that this object-point association is also used when computing object-point measurement error during BA in (11). Finally, we match two objects in different frames if they have the most number of shared feature points between each other and the number also exceeds certain threshold (10 in our implementation). Through our experiments, this approach works well for wide baseline matching, repetitive objects, and occlusions. The dynamic feature points belonging to moving objects are discarded because they cannot fulfill the epipolar constraint. Therefore, objects with few associated feature points are considered as dynamic objects, for example, the front cyan car in Fig. 5.

V. DYNAMIC SLAM

The previous section deals with the static object SLAM. In this section, we propose an approach to jointly estimate the camera pose and dynamic object trajectories. Our approach makes some assumptions about the objects to reduce the number of unknowns and make the problem solvable. The two commonly used assumptions are that the object is rigid and follows some physically feasible motion model. The rigid body assumption indicates that a point's position on its associated object does not change over time. This allows us to utilize the standard 3-D map point reprojection error to optimize its position. For the motion model, the simplest form is constant motion model with uniform velocity. For some specific object, such as vehicles, it is additionally constrained to follow the nonholonomic wheel model (with some side slip).

A. Notations

We define some new map elements in addition to the static SLAM in Section IV-A. For the dynamic object O^i , we need to estimate its pose ${}^jO^i$ in each observed frame j . We use “dynamic points” to refer to feature points associated with moving objects. For dynamic point P^k on moving object O^i , we represent its position anchored on the object as ${}^iP^k$, which is fixed based on the rigidity assumption. Its world pose will change over time and is not suitable for SLAM optimization.

B. SLAM Optimization

The factor graph of the dynamic object estimation is shown in Fig. 6. Blue nodes are the static SLAM components, while the red ones represent the dynamic objects, points, and motion velocity. The green squares are the measurement factors, including the camera-object factor in (10), the object-velocity factor in (14), and the point-camera-object factor in (15) which will be explained as follows. With these factors, camera poses can also be constrained by the dynamic elements.

1) *Object Motion Model*: The general 3-D object motion can be represented by a pose transformation matrix $T \in SE(3)$. We

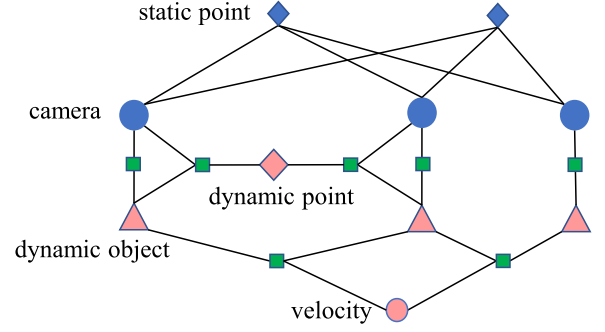


Fig. 6. Dynamic object SLAM. Blue nodes represent the static SLAM component and red ones represent new dynamic variables. The green squares are the new factors of dynamic map, including motion model constraints and observations of objects and points.

can apply T to the previous pose and then compute the pose error with the current pose. Here, we adopt a more restricted nonholonomic wheel model [39] that is also used in other dynamic vehicle tracking work [14]. Car motion is represented by linear velocity v and steering angle ϕ . Suppose the vehicle moves on a local planar surface approximately, then object roll/pitch $=0$ and translation in z axis $t_z = 0$. Only t_x, t_y, θ (heading yaw) is needed to represent its full state $T_o = [R(\theta) [t_x, t_y, 0]^T]$. The predicted state from velocity is

$$\begin{bmatrix} t'_x \\ t'_y \\ \theta' \end{bmatrix} = \begin{bmatrix} t_x \\ t_y \\ \theta \end{bmatrix} + v\Delta t \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \\ \tan(\phi)/L \end{bmatrix} \quad (13)$$

where L is the distance between the front and rear wheel center. Note that this model requires that x, y, θ is defined at the rear wheel center, while our object frame is defined at the vehicle center. The two coordinate frames have $L/2$ offset that needs to be compensated. The final motion model error is simply as

$$e_{mo} = [t'_x, t'_y, \theta'] - [t_x, t_y, \theta]. \quad (14)$$

2) *Dynamic Point Observation*: As explained before, the dynamic point is anchored to its associated object, so it is first transformed to the world frame, and then projected onto the camera. Suppose the local position of k th point on i th object is ${}^iP^k$ and the object pose in the j th image is ${}^jT_o^i$, then the point's reprojection error is

$$e_{dp} = \pi({}^jT_o^i {}^iP^k, T_c^j) - z_{kj} \quad (15)$$

where T_c^j is the j th camera pose and z_{kj} is the observed pixel of this point.

C. Dynamic Data Association

Through the experiments, we find that the association method for static environments in Section IV-C is not suitable for the dynamic cases due to the difficulty in matching dynamic point features. The typical way to track a feature point is to predict its projected position, search nearby features that match descriptors, and then check epipolar geometry constraints [4]. However, for monocular dynamic cases, it is difficult to accurately predict the movement of objects and points and the epipolar geometry is also not accurate when object motion is inaccurate.

Thus, we designed different approaches for the point and object association. The feature points are directly tracked by the 2-D KLT sparse optical flow algorithm, which does not require the 3-D point position. After pixel tracking, the 3-D position of the dynamic features will be triangulated considering the object movement. Mathematically, suppose the projection matrix of two frames are M_1 and M_2 . The 3-D point positions in these two frames are P_1, P_2 and corresponding pixel observations are z_1, z_2 . The object movement transformation matrix between two frames is ΔT , then we can infer that $P_2 = \Delta T P_1$. Based on projection rule, we have

$$\begin{aligned} M_1 P_1 &= z_1 \\ M_2 \Delta T P_1 &= z_2. \end{aligned} \quad (16)$$

If we treat $M_2 \Delta T$ as a modified camera pose compensating object movement, the abovementioned equation is the standard two-view triangulation problem [35] that can be solved by SVD.

KLT tracking might still fail when the pixel displacement is large, for example, when another vehicle comes close and toward the camera. Therefore, for the dynamic object tracking, we do not utilize the shared feature point matching approach in Section IV-C. Instead, we directly utilize visual object tracking algorithm [40]. The object's 2-D bounding box is tracked and its position is predicted from the previous frame, then it is matched to the detected bounding box in the current frame with the largest overlapping ratio.

VI. IMPLEMENTATIONS

A. Object Detection

For the 2-D object detection, we use the YOLO detector [41] with a probability threshold of 0.25 for indoor scenarios and MS-CNN [42] with a probability of 0.5 for outdoor KITTI. Both run in real time on a GPU.

If an accurate camera pose is known, for example, in the SUN RGBD dataset, we only need to sample the object yaw to compute the VPs as explained in Section III-A4. Fifteen samples of the object yaw in a range of 90° are generated as cuboids that can be rotated, as mentioned in Section IV-B1. Then ten points are sampled on the top edge of the 2-D bounding box. Note that not all the samples can form valid cuboid proposals, as some cuboid corners might lie outside of the 2-D box. In scenarios with no ground truth camera pose provided, we sample camera roll/pitch in a range of $\pm 20^\circ$ around the initially estimated angle. For single images with no prior information, we simply estimate that camera is parallel to ground. For multiview scenarios, SLAM is used to estimate the camera pose. One advantage of our approach is that it does not require large training data as we only need to tune the two cost weights in (4). It can also run in real time, including the 2-D object detection and edge detection.

B. Object SLAM

The pipeline of the whole SLAM algorithm is shown in Fig. 4(a). As mentioned in Section IV, our system is based on ORB SLAM2 and we did not change the camera tracking and

keyframe creation modules. For the newly created keyframe, we detect the cuboid objects, associate them, then perform BA with camera poses and points. For the dynamic objects, we can choose to reconstruct or ignore them depending on different tasks. The cuboid is also used to initialize the depth for feature points that are difficult to triangulate when stereo baseline or parallax angle is smaller than a threshold. This can improve the robustness in some challenging scenarios, such as large camera rotations, as demonstrated in the experiments. Since the number of objects is far less than points, object association, and BA optimization runs efficiently in real time. To get an absolute map scale for monocular SLAM, the initial frame's camera height is provided to scale the map. Note that our object SLAM can also work independently without points. In some demonstrated challenging environments with few feature points, ORB SLAM cannot work, but our algorithm can still estimate camera poses using only the object-camera measurement.

There are different costs in the optimization (see in Section IV) and some of them are in pixel space, for example (10), while some are in Euclidean space, such as (8) and (11), therefore it is necessary to tune the weights between them. Since it is difficult to analyze the cuboid detection uncertainty, we mainly hand tune the object cost weights by inspecting the number and magnitude of measurement so that different types of measurements contribute roughly the same. For example, there are only a few objects compared to points, but their reprojection error in (10) is much larger compared to points. From our experiments, object-camera and point-camera measurements have similar weights.

C. Dynamic Object

The implementation of dynamic objects mostly follows the previous section with some difference. The constant motion model assumption may not hold for practical datasets because objects may accelerate and decelerate (for example in Fig. 13). Through the ground truth object velocity analysis, we find that the velocity roughly stays the same in about 5 s. Therefore, in our SLAM, motion model constraint is only applied to observations in the last 5 s.

VII. EXPERIMENTS—SINGLE VIEW DETECTION

The SUN RGBD [43] and KITTI object [44] data with ground truth 3-D bounding box annotations are used for single view object detection evaluation. 3-D intersection over union (IoU) and average precision (AP) is adopted as the evaluation metric instead of only rotation or viewpoint evaluation in many other works. As there is no depth data, the 3-D IoU threshold for a correct detection is adjusted to 25% [12], [43]. Since our approach does not depend on the prior object model, in order to get an absolute scale of object position and dimensions, we only evaluate the ground objects with known camera height, as explained in Section III-A. For the KITTI dataset, this assumption is already satisfied. The commonly used training and validation index split by [2], [13] is used. For the SUN RGBD dataset, we select 1670 images with a visible ground plane and ground objects fully in the field of view.

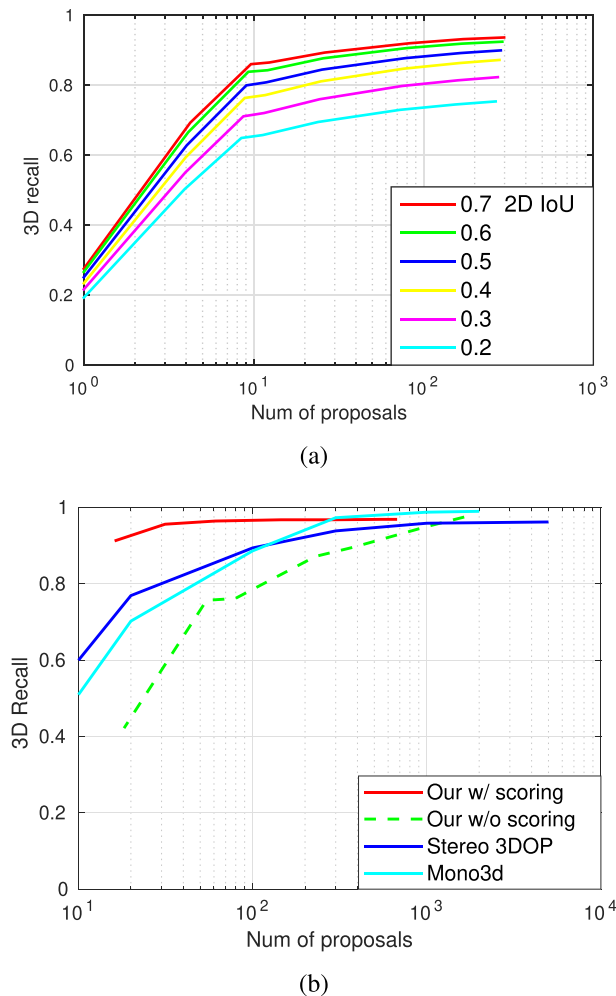


Fig. 7. (a) 3-D proposal recall on SUN RGBD Subdataset. Different lines correspond to different 2-D box IoU for association. (b) 3-D proposal recall on KITTI dataset. Our approach can get higher recall with fewer proposals.

A. Proposal Recall

We first evaluate the quality of proposal generation in SUN RGBD. It is obvious that if the 2-D bounding box is inaccurate, our 3-D cuboid accuracy will also be affected. This effect is analyzed by evaluating the 3-D recall on objects with a 2-D IoU greater than a threshold τ , as shown in Fig. 7(a). As expected, a larger τ leads to a higher 3-D recall. Our approach can achieve 3-D recall of 90% using around 50 cuboid proposals when 2-D IoU is 0.6.

We then evaluate and compare the proposal quality on the KITTI dataset shown in Fig. 7(b). Since Mono3D [12] and 3DoP [45] utilize different validation indexes compared to us, we only evaluate on the common images (1848). From our tests, different image indexes only lead to small result changes. Results of other algorithms are taken from their paper. Note that Mono3D first exhaustively samples huge amounts of cuboid proposals (~ 14 k), and then reports the recall after scoring and selecting the top proposals based on semantic and instance segmentation. Therefore, we also evaluate the recall before and after scoring. Before scoring (green line), our approach can reach a recall of 90% with 800 raw proposals per image, about 200 proposals

TABLE I
COMPARISON OF 3-D OBJECT DETECTION ON SUN RGBD SUBSET AND KITTI DATASET

	Method	3D IoU	AP
SUN RGBD	Primitive [10]	0.36	0.27
	3dgp [46]	0.42	0.22
	Ours	0.39	0.27
	Ours*	0.45	0.30
KITTI	Deep3D [13]	0.33	0.69
	SubCNN [2]	0.21	0.17
	Mono3D [12]	0.22	0.27
	Ours	0.21	0.29
	Ours top 10	0.38	0.75

*On 3DGP detected images.



Fig. 8. Single image 3-D object detection examples in SUN RGBD and KITTI dataset.

per object. After scoring (red line), we can reach the same recall using just 20 proposals, much fewer compared to [12]. There are two main reasons for this. First, our 3-D proposals are of high quality because they are guaranteed to match the 2-D detected box. Second, our more effective scoring function. Note that our approach has an upper limit as shown in Fig. 7(b) because the 2-D detector might miss some objects.

B. Final Detection

We then evaluate the final accuracy of the best selected proposal. To the best of our knowledge, there is no trained 3-D detection algorithm in SUN RGBD. Therefore, we compared with two public approaches, SUN primitive [10] and 3-D Geometric Phrases (3DGP) [46], both of which are model-based algorithms like ours. Additionally, 3DGP uses fixed prior object models. We modify their code to use the actual camera pose and calibration matrix when detecting and unprojecting to 3-D space.

For 3-D IoU evaluation, in order to eliminate the effect of 2-D detector, we only evaluate 3-D IoU for objects with 2-D rectangle $\text{IoU} > 0.7$. As shown in Table I and Fig. 8, our approach can generate many more accurate cuboids. Other approaches [10], [46] can only detect around 200 cuboids in the SUN RGBD subset datasets while our algorithm detects ten times more. Our

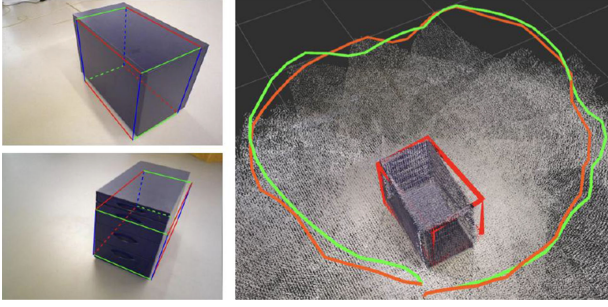


Fig. 9. Object SLAM on TUM fr3_cabinet. Red cube is the optimized object landmark, matching well with the ground truth point cloud. Red and green trajectories are the predicted and ground truth camera paths. Existing SLAM algorithms fail on this dataset due to low texture. Only object is used in SLAM.

TABLE II
OBJECT DETECTION AND SLAM RESULT ON INDOOR DATASETS

Dataset	Object IoU		Pose error (m)		
	single view	after BA	DSO *	ORB *	Our
fr3 cabinet	0.46	0.64	—	—	0.17
ICL room2	0.33	0.49	0.01	0.02	0.03
Two Chair	0.37	0.58	0.01	—	0.01
Rot Chair	0.35	0.50	—	—	0.05

*Pose error with scale alignment.

mean 3-D object IoU is smaller compared 3DGP using prior models, but higher if we only evaluate on the same detected images (≈ 200) by 3DGP. Similarly for AP, we evaluate other methods only on the images where they detect 3-D objects, otherwise their AP value will be very low ($< 5\%$) compared to our 27% on the whole dataset.

On the KITTI dataset, we compare with other monocular algorithms [2], [12], [13] using deep networks. SubCNN additionally uses prior models. Prediction results are provided by their authors. AP is evaluated on the moderate car class. From Table I, our approach performs similarly to SubCNN and Mono3D. As SubCNN generates many false positive detections, their AP value is low. The best performing approach is Deep3D [13] which directly predicts vehicle orientations and dimension using deep networks. As there is only one object class “car” with fixed camera poses and object shapes, CNN prediction works better than our hand-designed features. The last row is the evaluation of our selected top ten cuboid proposals to show that our proposal generation part can still generate high quality proposals.

VIII. EXPERIMENTS—OBJECT SLAM

We then evaluate the performance of object SLAM, including camera pose estimation, and 3-D object IoU after BA optimization. We show that SLAM and object detection can benefit each other in various datasets. Root mean squared error (RMSE) [47] and KITTI translation error [44] are used to evaluate the camera pose. Note that even though our algorithm is monocular SLAM, we can get the map scale from the first frame’s camera height, therefore, we directly evaluate the absolute trajectory error without aligning it in scale. To better evaluate the monocular pose drift, we turn OFF the loop closure module in ORB SLAM when using and comparing with it.

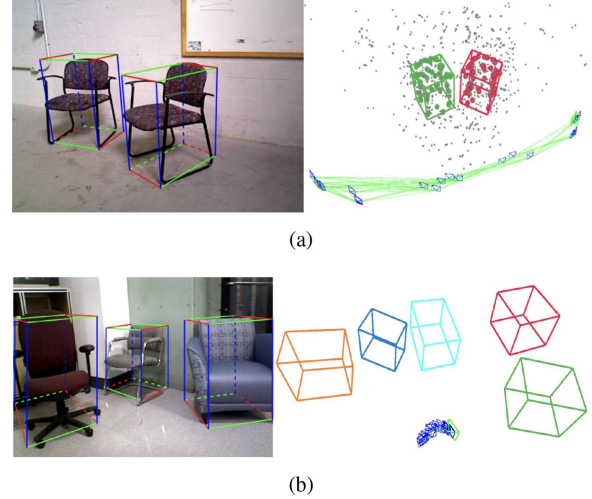


Fig. 10. Collected chair datasets. (a) Objects fit tightly with the associated points after BA optimization. (b) Objects can improve camera pose estimation when there is large camera rotation.

A. TUM RGBD and ICL-NUIM Dataset

These datasets [47], [48] have ground truth camera pose trajectory for evaluation. We only use the RGB images for the SLAM algorithm. For ground truth of the objects, 3-D cuboids are manually labeled in a registered global point cloud from the depth images.

We first test on TUM fr3_cabinet, shown in Fig. 9, which is a challenging low texture dataset, and existing monocular SLAM algorithms all fail on it due to few point features. The object is the only SLAM landmark and the 3-D object-camera measurement in Section IV-B1 is used because it can provide more constraints than 2-D measurement. The left of Fig. 9 shows our online detected cuboid in some frames using estimated camera pose from SLAM. There is clearly large detection error in the bottom image. After multiview optimization, the red cube in the map almost matches with the ground truth point cloud. From row “fr3/cabinet” in Table II, 3-D object IoU is improved from 0.46 to 0.64 after SLAM optimization compared to the single image cuboid detection. The absolute camera pose error is 0.17 m.

We then test on the ICL living room dataset, which is a general feature rich scenario. Since there is no absolute scale for monocular DSO or ORB SLAM, we compute their pose error after scale alignment [15]. We improve the object detection accuracy while sacrificing some camera pose accuracy due to imperfect object measurements. As can be seen from the mapping result of ICL data in Fig. 1(a), our approach is able to detect different objects, including sofas, chairs, and pot plant, demonstrating the advantage of our 3-D detection without prior models.

B. Collected Chair Dataset

We collect two chair datasets using a Kinect RGBD camera shown in Fig. 10. The RGBD ORB SLAM result is used as the ground truth camera poses. The second dataset contains large camera rotation, which is challenging for most monocular SLAM. As shown in Fig. 10(a), after optimization, cuboids

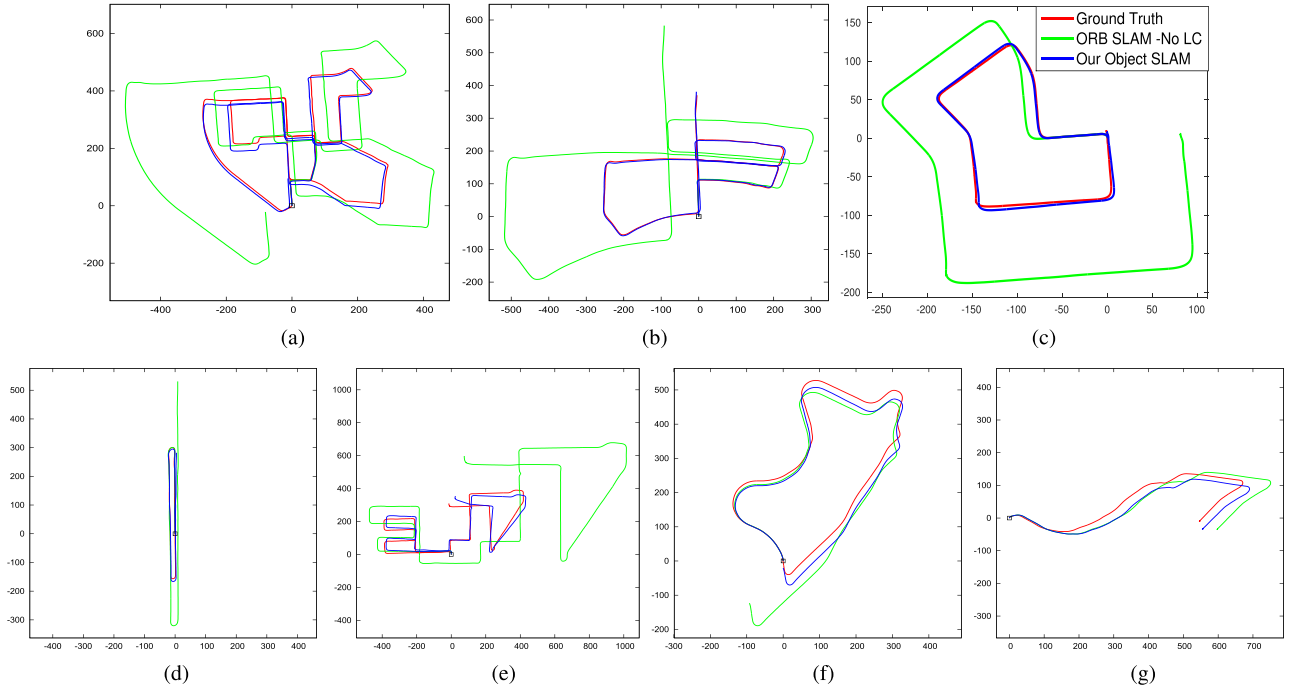


Fig. 11. Our object SLAM on KITTI odometry benchmark without loop closure detection and constant ground height assumption. Red is ground truth pose estimation. Blue is our object SLAM result. Green is ORB SLAM without loop detection. Objects can reduce monocular scale drift and improve pose estimation accuracy. (a) Sequence 00, (b) 05, (c) 07, (d) 06, (e) 08, (f) 09, and (g) 10.

can fit the associated 3-D points tightly showing that object and point estimation benefit each other. The quantitative error is shown in the bottom two rows of Table II. DSO is able to work in the first dataset, but performs poorly in the second one, due to the large camera rotation. Mono ORB SLAM fails to initialize in both cases while our cuboid detection can provide depth initialization for points even from a single image. Similar as before, the 3-D object IoU is also improved after BA.

C. KITTI Dataset

We tested on two of the KITTI datasets, the short sequence, with provided ground truth object annotations, and the long sequence, which is a standard odometry benchmark without object annotations. The 2-D object-camera measurement in Section IV-B1 is used for BA because of its low uncertainty compared to 3-D measurements for vehicle detection. We also scale ORB SLAM's initial map by the first frame camera height (1.7 m in our implementations) in order to evaluate its absolute pose error. In Fig. 11, we can observe that the initial trajectory segment before first turning matches well with ground truth, indicating the initial map scaling for ORB is correct. For KITTI dataset, we additionally initialize object dimension using prior car size ($w = 3.9, l = 1.6, h = 1.5$ in our implementation) to maintain long-term scale consistency, which is also used in other object SLAM works [21], [22]. This is especially useful when objects are not observed frequently in some sequence.

1) *KITTI Raw Sequence*: We select ten KITTI raw sequences with the most number of ground truth object annotations, named "2011_0926_00xx." The ground truth camera pose is from the provided GPS/INS poses on KITTI. For the

object IoU, we compare three methods. The first is the single image cuboid detection [13]. The second is the object pose just using SLAM data association between frames, shown as row "Ours before BA." For example, if an object in one frame is far away, the 3-D detection may be inaccurate, but in another frame, the same object is closer thus the 3-D detection becomes more accurate. Therefore, data association with correct camera pose estimation should also improve 3-D detection. Third, the object poses after our final BA optimization are also evaluated shown as row "Ours."

As shown in the top three rows of Table III, object accuracy is increased after data association and BA optimization in most of the sequences, however, in some sequences, due to local position drift, the object IoU may also decrease a bit. For camera pose estimation, object SLAM can provide geometry constraints to reduce the scale drift of monocular SLAM. Note that since most KITTI raw sequences do not have loops, disabling or enabling the ORB SLAM loop closure module does not make a difference.

2) *KITTI Odometry Benchmark*: Most existing monocular SLAMs use a constant ground plane height assumption on the benchmark to reduce monocular scale drift [49], [50]. Recently, there are also some object-based scale recovery approaches [21], [22]. Their results are directly taken from their papers. Similar to other approaches, we did not compare with ORB SLAM in this case, as without loop closure, it cannot recover scale in the long sequence and has significant drift error shown in Fig. 11. As shown in Table IV, our object SLAM achieves 2.74% translation error and performs much better than other SLAM using objects. This is because they represent vehicles as spheres or only use vehicle height information, which is not as

TABLE III
OBJECT DETECTION AND CAMERA POSE ESTIMATION ON KITTI RAW SEQUENCE

Seq		22	23	36	39	61	64	93	95	96	117	Mean
Object 3D IoU	Single view [13]	0.52	0.32	0.50	0.54	0.54	0.43	0.43	0.40	0.26	0.25	0.42
	Ours before BA	0.55	0.36	0.49	0.56	0.54	0.42	0.46	0.49	0.20	0.30	0.44
	Ours	0.58	0.35	0.54	0.59	0.50	0.48	0.45	0.52	0.29	0.35	0.47
Trans error(%)	ORB -No LC	13.0	1.17	7.08	6.76	1.06	7.07	4.40	0.86	3.96	4.10	4.95
	Ours	1.68	1.72	2.93	1.61	1.24	0.93	0.60	1.49	1.81	2.21	1.62

TABLE IV
CAMERA POSE ESTIMATION ERROR ON KITTI ODOMETRY BENCHMARK

Seq			0	2	3	4	5	6	7	8	9	10	Mean
Trans Error (%)	Ground based	[49]	4.42	4.77	8.49	6.21	5.44	6.51	6.23	8.23	9.08	9.11	6.86
		[50]	2.04	1.50	3.37	1.43	2.19	2.09	—	2.37	1.76	2.12	2.03
		Ours	1.83	2.11	2.55	1.68	2.45	6.31	5.88	4.2	3.37	3.48	3.39
	Object based	[22]	3.09	6.18	3.39	32.9	4.47	12.5	2.81	4.11	11.2	16.8	9.75
		Ours	2.40	4.25	2.87	1.12	1.64	3.20	1.63	2.79	3.16	4.34	2.74
	Combined	Ours	1.97	2.48	1.62	1.12	1.64	2.26	1.63	2.05	1.66	1.46	1.78
RMSE (m)	Object based	[21]	73.4	55.5	30.6	10.7	50.8	73.1	47.1	72.2	31.2	53.5	49.8
		Ours	13.9	26.2	3.79	1.10	4.75	6.98	2.67	10.7	10.7	8.37	8.91

accurate as our cuboid BA. Our algorithm is also comparable to ground-based scaling approaches. Visualization of some object mapping and pose estimation are shown in Figs. 1(b) and 11, where we can see that our approach greatly reduces monocular scale drift.

Our object SLAM performs worse in some sequences, such as Seq 02, 06, 10, mainly because there are not many objects visible over long distance causing large scale drift. Therefore, we also propose a simple method to combine ground height assumption with our object SLAM. If there are no objects visible in recent 20 frames, we fit a ground plane from point cloud then scale camera poses and local map based on the constant ground height assumption. As shown in Table IV, the fourth row “ground based/Ours” representing ORB SLAM only with our simple ground scaling and without objects, achieves good performance with 3.39% translation error. The row “Combined,” which combines ground scaling with objects, further reduces the error to 1.78% and achieves the state-of-art accuracy of monocular SLAM on the KITTI benchmark. Note that ground plane based approaches also have their limitations, for example, they will not work for aerial vehicle or handheld cameras. They will also fail when the ground is not visible, such as frames in Fig. 5 of KITTI 07. The front dynamic car occludes the ground for a long time and therefore many ground-based approaches fail or perform poorly on KITTI 07.

D. Dynamic Object

We also test the algorithm on the dynamic car sequences on the KITTI datasets as shown in Fig. 12(a). We select some raw sequences with more dynamic objects observed over a long time shown in Table V. The full name of the sequences is “2011_0926_00xx.” The first four sequences also correspond to Seq 3,4,5,18 in KITTI tracking dataset. Most cars in these sequences are moving. Ground truth object annotations are available for all or some frames and ground truth camera poses are also provided by GPS/INS.

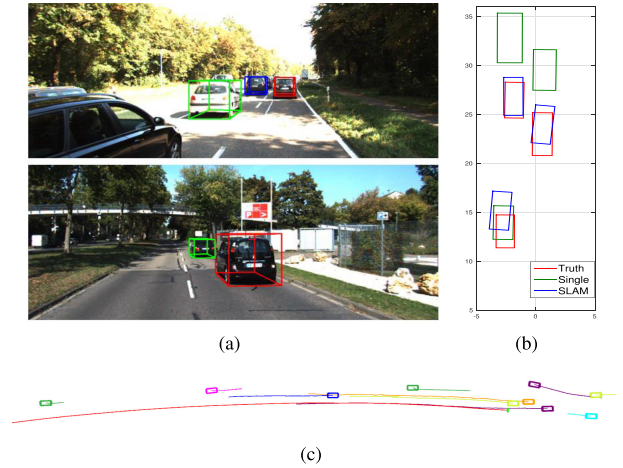


Fig. 12. Dynamic object SLAM result on KITTI. (a) Sample frames of single image cuboid detection. (b) Top view comparison of 3-D detections in single image and multiview SLAM. (c) Camera and object pose estimation. The red curve starting from left is the camera’s trajectory. Other curves attached with rectangle markers represent the dynamic object’s trajectory.

1) *Qualitative Results:* Some single image detection examples are shown in Fig. 12(a). Fig. 12(b) shows the top view of the first image of Fig. 12(a). For the two distant front cars, even though the 2-D image cuboid detection looks good, it actually has a large 3-D distance error. This is because only the car’s back faces are observable causing ill constraint single image detection. After multiview dynamic object BA, the blue optimized object matches better with the ground truth, mostly due to the motion model constraints. However, it can sometimes decrease the accuracy, for example, the bottom object in the figure. Some possible reasons are due to the noisy 2-D and 3-D object detection, especially for the close objects. The constant motion assumption may also cause errors when the vehicle accelerates or decelerates. Fig. 12(c) shows all the dynamic objects’ history poses as well as camera poses. The objects’ trajectories are smooth due to the motion model constraints.

TABLE V
DYNAMIC OBJECT DETECTION AND CAMERA POSE ESTIMATION ON KITTI RAW SEQUENCE

Seq		13	14	15	04	56	32	Mean
Object	Single view	0.41	0.11	0.42	0.11	0.54	0.34	0.32
3D IoU	Ours	0.51	0.28	0.44	0.42	0.42	0.26	0.39
Trans	ORB-No LC	2.34	11.5	2.4	2.31	8.45	2.76	4.96
error(m)	Ours	0.99	7.62	1.94	1.50	5.39	3.07	3.42

TABLE VI
DYNAMIC OBJECT LOCALIZATION COMPARISON ON KITTI RAW SEQUENCE

Seq		04				47				56	Mean
Obj. ID		1	2	3	6	0	4	9	12	0	
No. Frames		91	251	284	169	170	96	94	637	293	
Depth error (%)	[51]	4.1	6.8	5.3	7.3	9.6	11.4	7.1	10.5	5.5	7.9
	[32]	6.0	5.6	4.9	5.9	5.9	12.5	7.0	8.2	6.0	6.8
	Ours	5.1	11.1	1.8	2.5	9.3	3.8	6.6	2.8	0.8	4.9

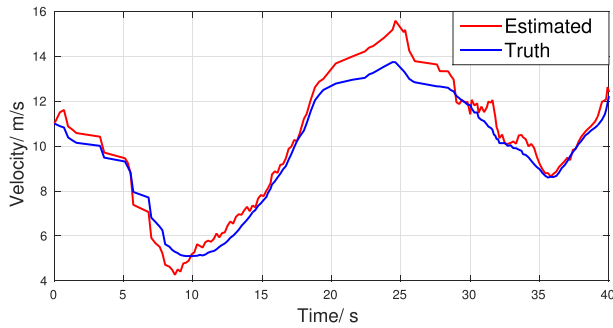


Fig. 13. Dynamic car velocity estimation on KITTI Seq 0047. Our SLAM algorithm based on the piecewise constant motion assumption can correctly estimate the moving object's velocity using just a monocular camera.

Fig. 13 shows the velocity estimation of one of objects in on Seq 0047 data. We can see that the computed ground truth object velocity also changes with time, therefore, the piecewise constant velocity motion explained in Section VI-C is reasonable. With a monocular camera, the proposed algorithm can roughly estimate the object's absolute velocity.

2) *Quantitative Results:* Since there is no current monocular SLAM utilizing dynamic objects to change camera pose estimation, we thus directly compare with the state-of-the-art featured based ORB SLAM. Though it already has some modules to detect dynamic points as outliers based on reprojection error, to compare with ORB SLAM fairly, we directly remove features lying in the dynamic object areas and report its pose estimation result. From Table V, we can see that our method can improve the camera pose estimation on most sequences, especially when objects can be observed and tracked over many consecutive frames, for example, in the first four datasets. This is because with more observations, the objects' velocity and dynamic points' position can be estimated more accurately and thus have more effect on the camera pose estimation, while in the last two sequences, objects are usually observed by only a few frames.

We also compare the 3-D object localization with other monocular methods shown in Table VI. The most similar one to us is [32], which utilized semantic and geometric costs to optimize object locations, but their approach assumed that the

TABLE VII
RUNTIME OF DIFFERENT SYSTEM COMPONENTS

Dataset	Tasks	Runtime (mSec)
Outdoor KITTI	Tracking thread (per frame)	33.0
	No object BA	182.7
	Static object BA	194.5
	Dynamic object BA	365.2
Indoor ICL room	Tracking thread (per frame)	15.0
	No object BA	49.5
	Static object BA	55.3

camera poses are already solved and fixed. We utilize the same metric in [32] to measure the relative object depth error in each frame and the number is taken directly from the paper. From the table, our method outperforms others on most sequences. In the sequence 56, the relative depth error is only 0.8%.

E. Time Analysis

Finally, we provide the computational analysis of our system. The experiments are carried out on Intel i7-4790 CPU at 4.0 GHz. The 2-D object detection time depends on the GPU power and CNN model complexity. Many algorithms, such as Yolo, can run in real time. After getting the 2-D boxes, our 3-D cuboid detection takes about 20 ms per image and the main computation of it is edge detection. For the SLAM part implemented in C++ on CPU, we show the time usage in Table VII. The computation strongly depends on the datasets with different image size and textures, therefore we choose two representative sequences: Outdoor KITTI 07 at 10 Hz shown in Fig. 1(b) and indoor ICL-NUIM livingroom dataset at 30 Hz shown in Fig. 1(a). On average, there are five object landmarks in each local BA optimization in the two datasets. The tracking thread includes the ORB point feature detection and camera pose tracking for each frame which can run in real time from the table. The BA map optimization occurs when a new keyframe is created, therefore it does not need to run in real time. We show the time usage of BA when different types of landmarks exist. In the static environment, adding objects into the system only increases the optimization by 7%. This is reasonable as there are only a few objects in the local map optimization. For the

dynamic cases, due to many new variables and measurements of dynamic points, the computation increases by a factor of two.

IX. CONCLUSION

In this paper, we conveyed a general approach for monocular 3-D object detection and SLAM mapping without prior object models, in both static and dynamic environments. More importantly, we demonstrated for the first time, that semantic object detection and geometric SLAM could benefit each other in one unified framework.

For the single image 3-D object detection, we proposed a new method to efficiently generate high quality cuboid proposals from the 2-D bounding box based on VP. Proposals were then scored efficiently by image cues. In the SLAM part, we proposed an object level SLAM with novel measurement functions between cameras, objects, and points, and new object association methods to robustly handle occlusion and dynamic movement. Objects can provide long range geometric and scale constraints for camera pose estimation. In turn, SLAM also provided camera pose initialization for detecting and refining 3-D object. For the dynamic scenarios, we also showed that with the new measurement constraints, the moving object and point can also improve the camera pose estimation through the tightly coupled optimization.

We evaluate the two parts on different indoor and outdoor datasets and achieve the best accuracy of 3-D object detection on SUN RGBD subset data and camera pose estimation on KITTI odometry datasets. In the future, we are also interested in the dense mapping using objects. More complete scene understanding can also be integrated with SLAM optimization.

REFERENCES

- [1] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2961–2969.
- [2] Y. Xiang, W. Choi, Y. Lin, and S. Savarese, "Subcategory-aware convolutional neural networks for object proposals and detection," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2017, pp. 924–933.
- [3] F. Chabot, M. Chaouch, J. Rabarisoa, C. Teulière, and T. Chateau, "Deep MANTA: A coarse-to-fine many-task network for joint 2D and 3D vehicle analysis from monocular image," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2040–2049.
- [4] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM: A versatile and accurate monocular SLAM system," *IEEE Trans. Robot.*, vol. 31, no. 5, pp. 1147–1163, Oct. 2015.
- [5] S. Yang and S. Scherer, "Direct monocular odometry using points and lines," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 3871–3877.
- [6] M. Kaess, "Simultaneous localization and mapping with infinite planes," in *Proc. Int. Conf. Robot. Autom.*, 2015, pp. 4605–4611.
- [7] J. K. Murthy, G. V. S. Krishna, F. Chhaya, and K. M. Krishna, "Reconstructing vehicles from a single image: Shape priors for road scene understanding," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 724–731.
- [8] J. J. Lim, H. Pirsiavash, and A. Torralba, "Parsing IKEA objects: Fine pose estimation," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2013, pp. 2992–2999.
- [9] W. Kehl, F. Manhardt, F. Tombari, S. Ilic, and N. Navab, "SSD-6D: Making RGB-based 3D detection and 6D pose estimation great again," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1521–1529.
- [10] J. Xiao, B. Russell, and A. Torralba, "Localizing 3D cuboids in single-view images," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 746–754.
- [11] V. Hedau, D. Hoiem, and D. Forsyth, "Thinking inside the box: Using appearance models and context based on room geometry," in *Proc. Eur. Conf. Comput. Vis.*, 2010, pp. 224–237.
- [12] X. Chen, K. Kundu, Z. Zhang, H. Ma, S. Fidler, and R. Urtasun, "Monocular 3D object detection for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2147–2156.
- [13] A. Mousavian, D. Anguelov, J. Flynn, and J. Kosecka, "3D bounding box estimation using deep learning and geometry," *IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7074–7082.
- [14] P. Li, T. Qin, and S. Shen, "Stereo vision-based semantic 3D object and ego-motion tracking for autonomous driving," in *Proc. Eur. Conf. Comput. Vis.*, Sep. 2018, pp. 646–661.
- [15] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [16] J. Dong, X. Fei, and S. Soatto, "Visual-inertial-semantic scene representation for 3D object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 960–970.
- [17] S. Pillai and J. Leonard, "Monocular SLAM supported object recognition," in *Proc. Robot. Sci. Syst.*, 2015, pp. 34–42.
- [18] A. Dame, V. A. Prisacariu, C. Y. Ren, and I. Reid, "Dense reconstruction using 3d object shape priors," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 1288–1295.
- [19] S. Y. Bao, M. Bagra, Y.-W. Chao, and S. Savarese, "Semantic structure from motion with points, regions, and objects," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 2703–2710.
- [20] R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, "SLAM+: Simultaneous localisation and mapping at the level of objects," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 1352–1359.
- [21] D. Frost, V. Prisacariu, and D. Murray, "Recovering stable scale in monocular slam using object-supplemented bundle adjustment," *IEEE Trans. Robot.*, vol. 34, no. 3, pp. 736–747, Jun. 2018.
- [22] E. Sucar and J.-B. Hayet, "Bayesian scale estimation for monocular SLAM based on generic object detection for correcting scale drift," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018.
- [23] D. Gálvez-López, M. Salas, J. D. Tardós, and J. M. M. Montiel, "Real-time monocular object SLAM," *Robot. Auton. Syst.*, vol. 75, pp. 435–449, 2016.
- [24] C. Rubino, M. Crocco, and A. D. Bue, "3D object localisation from multi-view image detections," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 6, pp. 1281–1294, Jun. 2018.
- [25] L. J. Nicholson, M. J. Milford, and N. Sunderhauf, "QuadricSLAM: Dual quadrics from object detections as landmarks in object-oriented SLAM," *IEEE Robot. Autom. Lett.*, vol. 4, no. 1, pp. 1–8, Jan. 2019.
- [26] S. L. Bowman, N. Atanasov, K. Daniilidis, and G. J. Pappas, "Probabilistic data association for semantic SLAM," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 1722–1729.
- [27] S. Yang, Y. Song, M. Kaess, and S. Scherer, "Pop-up SLAM: A semantic monocular plane SLAM for low-texture environments," in *Proc. IEEE Int. Conf. Intell. Robot. Syst.*, 2016, pp. 1222–1229.
- [28] N. Yang, R. Wang, J. Stückler, and D. Cremers, "Deep virtual stereo odometry: Leveraging deep depth prediction for monocular direct sparse odometry," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 835–852.
- [29] S. Wang, R. Clark, H. Wen, and N. Trigoni, "Deepvo: Towards end-to-end visual odometry with deep recurrent convolutional neural networks," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 2043–2050.
- [30] I. A. Bârsan, P. Liu, M. Pollefeys, and A. Geiger, "Robust dense mapping for large-scale dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 7510–7517.
- [31] B. Bescós, J. M. Fácil, J. Civera, and J. Neira, "DynaSLAM: Tracking, mapping and inpainting in dynamic scenes," *IEEE Robot. Automat. Lett.*, vol. 3, no. 4, pp. 4076–4083, Oct. 2018.
- [32] S. Song and M. Chandraker, "Joint SFM and detection cues for monocular 3D localization in road scenes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3734–3742.
- [33] N. D. Reddy, P. Singhal, V. Chari, and K. M. Krishna, "Dynamic body VSLAM with semantic constraints," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2015, pp. 1897–1904.
- [34] M. Henein, G. Kennedy, R. Mahony, and V. Ila, "Exploiting rigid body motion for SLAM in dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018.
- [35] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge, U.K.: Cambridge Univ. Press, 2003.
- [36] R. G. von Gioi, J. Jakubowicz, J.-M. Morel, and G. Randall, "LSD: A fast line segment detector with a false detection control," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 4, pp. 722–732, Apr. 2010.
- [37] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: A general framework for graph optimization," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2011, pp. 3607–3613.
- [38] M. Kaess, A. Ranganathan, and F. Dellaert, "iSAM: Incremental smoothing and mapping," *IEEE Trans. Robot.*, vol. 24, no. 6, pp. 1365–1378, Dec. 2008.

- [39] S. M. LaValle. *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.
- [40] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 3, pp. 583–596, Mar. 2015.
- [41] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7263–7271.
- [42] Z. Cai, Q. Fan, R. Feris, and N. Vasconcelos, "A unified multi-scale deep convolutional neural network for fast object detection," in *Proc. Eur. Conf. Comput. Vision*, 2016, pp. 354–370.
- [43] S. Song, S. P. Lichtenberg, and J. Xiao, "SUN RGB-D: A RGB-D scene understanding benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 567–576.
- [44] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The kitti vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3354–3361.
- [45] X. Chen *et al.*, "3D object proposals for accurate object class detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 424–432.
- [46] W. Choi, Y.-W. Chao, C. Pantofaru, and S. Savarese, "Understanding indoor scenes using 3D geometric phrases," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2013, pp. 33–40.
- [47] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2012, pp. 573–580.
- [48] A. Handa, T. Whelan, J. B. McDonald, and A. J. Davison, "A benchmark for RGB-D visual odometry, 3D reconstruction and SLAM," in *Proc. IEEE Intl. Conf. Robot. Autom.*, May 2014, pp. 1524–1531.
- [49] B. Lee, K. Daniilidis, and D. D. Lee, "Online self-supervised monocular visual odometry for ground vehicles," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2015, pp. 5232–5238.
- [50] S. Song, M. Chandraker, and C. C. Guest, "High accuracy monocular SFM and scale correction for autonomous driving," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 4, pp. 730–743, Apr. 2016.
- [51] S. Song and M. Chandraker, "Robust scale estimation in real-time monocular SFM for autonomous driving," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2014, pp. 1566–1573.



Shichao Yang received the B.S. degree from Shanghai Jiao Tong University, China, in 2013, and the Ph.D. degree under the supervision of Sebastian Scherer at the Robotics Institute from Carnegie Mellon University, Pittsburgh, PA, USA, both in mechanical engineering.

He currently works as a Research Scientist with Facebook Reality Lab, Menlo Park, CA, USA. His research simultaneous localization and mapping (SLAM) and semantic scene understanding, with application to autonomous robots, and AR/VR.



Sebastian Scherer received the B.S. degree in computer science and the M.S. and the Ph.D. degrees in robotics from Carnegie Mellon University (CMU), Pittsburgh, PA, USA, in 2004, 2007, and 2010, respectively.

He is a Senior Systems Scientist with the Robotics Institute (RI) at CMU. He and his team have shown the fastest and most tested obstacle avoidance on a Yamaha RMax (2006), the first obstacle avoidance for microaerial vehicles in natural environments (2008), and the first (2010) and fastest (2014) automatic land-

ing zone detection and landing on a full-size helicopter. His research interest includes enabling autonomy for unmanned rotorcraft to operate at low altitude in cluttered environments.